The hurdy-gurdy Platform

Exploring the Frontier of Reducible Decidability in Practice

Christoph Kirsch
University of Salzburg, Austria
Czech Technical University, Prague, Czechia
ck@cs.uni-salzburg.at

Abstract

We are constructing a platform for exploring the *frontier of reducible decidability in practice* — which questions about programs can be answered, today, by chains of sound reductions to mechanized decision procedures within declared budgets — and, eventually, for pushing it. The method is *saturation*: present the platform a pinned benchmark and run it until every question is either answered along every feasible route, with measured cost and quantified trust, or open on the books with its first failing obstacle and the exact missing instrument — a *frontier pair* whose required contract is derived mechanically — or the stated reason none can. Each saturation deposits a map, and the maps compound. Throughout, LLMs both *deploy* hurdy-gurdy and *develop* it — playing the questions, and building the translation pairs and, in a gated shadow lane, the decision procedures the books demand — with user intervention minimized to one act, the valve: registering what may be built, or writing the mandate that delegates that registration. The architecture is what makes this safe: untrusted authors, trusted answers [12]. We state six facilitation theorems with their honest assumptions — exploration is sound (the map cannot be falsified by its untrusted explorer), monotone, self-directing, relatively complete in the lineage of Cook, terminating per benchmark, and domain-generic — with the compositional core mechanized in Lean 4, and close with the domain kit a field must supply and the pre-registered saturation protocol for a hardware model-checking corpus. Measurements are deliberately absent: the benchmarks section of this paper is the saturation report its protocol will produce.

1 Introduction

We are interested in constructing a platform that enables us to explore the frontier of *reducible decidability in practice* — and, eventually, to push it. The phrase is chosen word by word. A question about a program is *reducibly decidable* when some finite chain of sound reductions carries it to a mechanized decision procedure that answers it; the reachable set is the closure of the domain's questions under the reductions one actually has, not an oracle's reach. And *in practice* means within declared budgets, measured and never asymptotic: a procedure that exceeds every budget is not a way, and where that boundary lies is an empirical fact that moves with every solver, abstraction, and logic the world supplies. The frontier, so understood, is a real and interesting object: on its near side, everything formal methods can do today, enumerated and priced; on its far side, structured evidence about exactly what is missing. A platform that can *draw* that boundary — reliably, mechanically, in any domain that supplies the raw material — is a platform that can direct its own extension, because the far side of the map is the demand side for the instruments that do the pushing.

Saturating benchmarks is just a way to get us there. Present the platform a *benchmark*: a pinned, finite set of questions, designed preferably by someone else. Run the loop — ask, diagnose, book what fails, grow the instrument by exactly what the books demand, re-ask — until the benchmark *saturates*: every question either answered along every feasible route, with measured cost and quantified trust, or open on the books with its first failing obstacle, the evidence that would move it, and the exact missing instrument, or the honest statement that none can be named (Section 2). Saturation is a fixpoint, mechanically detected; the deposit is a *map* — the answered fraction per iteration, cost per answer, a way-census for every solved question, and a terminal board of *frontier pairs*: pairs whose design is unknown but whose required contract and design evidence are derived from the books (Definition 2.3). Maps compound, because the instruments built for one benchmark serve every later one.

The platform exists. hurdy-gurdy is a graph of deterministic, fidelity-graded translation pairs — built by untrusted LLM agents, gated by an architecture that makes their answers trustworthy anyway — and its calculus, platform, and measured July 2026 snapshot are the subject of a companion paper [12]. It is deployed the way it is developed: the player that asks, routes, and decides is an LLM, the builders that grow the graph are LLMs, and a saturation campaign is designed to run with user intervention contracted to one act — the valve of Section 4: registering what may be built, or writing the scoped mandate that delegates that registration one evidence-cited brief at a time. This paper reads that instrument from the story's end: every load-bearing feature is re-derived as a requirement of trustworthy cartography (Section 3), the growth loop as the explorer's engine (Section 4), and the whole as the subject of six *facilitation theorems* (Section 5): the map cannot be falsified by its untrusted explorer (F1), exploration never loses ground (F2), every open point

carries a unique, only-advancing diagnosis of what would extend the map there (F3), adequately targeted growth answers every answerable question — relative completeness, with fairness and gate liveness as the named assumptions (F4) — saturation terminates detectably (F5), and all of it is stated over an abstract domain signature, so it transfers to any domain supplying the kit of Section 6 (F6). The compositional core of the argument is mechanized in Lean 4, in the same no-dependency development as the instrument's calculus.

Contributions.

- The frontier problem, stated: answerability as an ordered five-condition filtration, the diagnosis as its first failure, saturation as a decidable per-benchmark fixpoint, and the map as the deliverable (Section 2).
- One currency for system state: implemented pairs carry *achieved* contracts, frontier pairs carry *required* contracts joined from the books, and routes — conditional routes included — compose both by one meet (Definitions 2.3 and 2.4).
- The facilitation theorems F1–F6 with proof burdens placed honestly — mechanized, reduced to named assumptions, or empirical — and necessity ablations showing each load-bearing feature of the instrument is needed (Section 5).
- The domain kit — the four obligations a field must meet for the theorems to apply — and the far side as a design oracle: per-obstacle extraction operators, a fragment atlas for shape-blocked demands, and terminal boards exported as challenge bundles (Section 6).
- The demand side extended past edges: a missing decision procedure is itself a typed generation target, classified *instantiation* or *discovery* by the atlas, and a candidate procedure — however authored — is admitted only through a dedicated gate (census replay against corroborated verdicts, canaries and verdict-flip mutants, certificate discipline, budget honesty); a synthesized in-process procedure is gated strictly harder than the pinned engines it would join (Section 4).
- A pre-registered saturation protocol for a hardware model-checking corpus designed by others, with the tooling built and exercised end to end — and no measurements: the benchmarks section of this paper is deliberately the saturation report its protocol will produce (Section 6).

Section 2 defines the problem. Section 3 and Section 4 compress the instrument and its loop to exactly what the theorems consume. Section 5 states and proves what can be proved. Section 6 says what pointing the explorer at a new domain costs, and pre-registers the first campaign.

2 The frontier problem

Domains. A *domain* is where questions live and decisions happen. Its signature supplies, and nothing else about it matters: languages, each carrying a *formal semantics* — a definable meaning function over its programs, exposing named *observables* — with formal semantics the only admission criterion; among them some *reasoning languages*, each owning mechanized decision procedures with declared budgets and, where available, certificate checkers; and a supply of independent *semantic anchors* — external formalizations against which a translation can be checked by something its author did not write. Hardware model checking is a domain; C program analysis is a domain; chemical reaction networks are a domain. The theorems of Section 5 use only the signature, which is what *in any given domain* will mean.

Questions. A *question* $q = (p, \varphi)$ is a program p in one of the domain's languages and a condition φ over p's observables, asked existentially (is φ reachable?) or universally (does φ hold for all inputs, within declared bounds?). A *benchmark* B is a pinned, finite set of questions with recorded provenance — pinned, so that "the benchmark" names the same set at every loop iteration.

Items, registries, candidates. The unit of admission is the *item*: either a *pair* — a deterministic, contract-carrying translation edge between two languages [12] — or a language-attached *capability*: a solver, a checker, an anchor. A *registry* G is the set of admitted items; growth is additive (items are admitted, never silently removed), and every admission is gated. A *candidate* answer for q rests on finitely many items: the route's hops plus the destination capabilities it invokes — one list. Pairs for edges, capabilities for nodes: a candidate is the route-level object, and it is the only object the definitions below need.

Definition 2.1 (Answerability)**.** Fix, per domain, N *conditions* on candidates, checked in a fixed *diagnosis order*. hurdy-gurdy instantiates $N = 5$: *connectivity* (the candidate is a route from p's language to a reasoning language), *loss* (φ's observables survive the route's composed projection), *shape* (φ's logical form is one the destination's procedures decide), *cost* (a procedure returns a verdict other than *unknown*/*resource-out* within its declared budget), and *trust* (the answer meets the asker's assurance floor). Write $S_k(G, q)$ for the candidates available in G that survive the first k conditions — the *answerability filtration* (Figure 1), shrinking as k grows and growing as G grows. Then q is *answerable at* G iff $S_N(G, q) \neq \emptyset$.

Two words in the title are fixed by this definition. *In practice*: the cost condition quantifies over declared budgets and measured runs, never over asymptotics — *resource-out* is a permanent verdict class, and a route without a measurement reads *unmeasured*, never cheap. *Reducible*: answerability quantifies over candidates built from admitted items, so

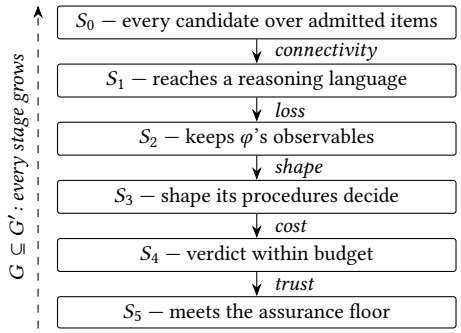

Figure 1. Answerability as an ordered filtration (Definition 2.1): candidates for (p, φ) shrink through the five conditions in diagnosis order; q is answerable at G iff $S_5(G, q) \neq \emptyset$. The diagnosis of an unanswerable question is the first condition whose stage empties (Definition 2.2), and since registry growth widens every stage, exploration never loses ground and the diagnosis only advances (Theorems 5.2 and 5.3).

the answerable set is the closure of the domain's questions under the admitted reductions — not an oracle's reach.

Definition 2.2 (Diagnosis)**.** For q unanswerable at G, the *diagnosis* is the first failing condition: the k with $S_k(G, q) \neq \emptyset = S_{k+1}(G, q)$. Theorem 5.3 shows it exists, is unique, and under growth only advances — "the obstacle" is honest vocabulary, and it is the frontier's local gradient: which kind of instrument would move the map *here*.

The currency: three tiers of pair. A pair carries a *contract*: its assurance class, the direction of its square, its kept observables, its measured cost [12]. Contracts compose along a route by the componentwise meet and are ordered by strength; the meet is what makes the following duality compose.

Definition 2.3 (Required and achieved contracts; the tiers)**.** An implemented pair carries an *achieved* contract, verified from above: evidence establishes *at least this*. A *frontier pair* carries a *required* contract, demanded from below: the join, over the open questions that cite it, of the observables they need kept, the direction and assurance they need, with cost kept as the histogram of their budgets. It also carries its *design evidence* — everything the books know about how to get there (Section 6) — and, where the demanded target is a reasoning language that does not exist, a *hypothetical* endpoint: a language sketch, a typed hole in the graph. The registry thus holds one lifecycle in three tiers:

$$\textit{frontier} \longrightarrow \textit{registered} \longrightarrow \textit{partial} \rightarrow \textit{built},$$

where *frontier* means design unknown, *registered* means a human admitted a design (the brief's "intended translator" is filled in), and *partial*/*built* mean an achieved, measured contract. Registering a frontier pair *means* exhibiting a design whose achievable contract dominates the required one; promotion only ever advances, and the evidence payload travels (Theorem 5.7).

Demand is not only edge-shaped. A missing *capability* — a decision procedure no registered hub supplies — is demanded in the same currency, and a curated atlas of the known decidability landscape (Section 6) draws its line against the known set: a *charted* procedure family (setting, family, and canonical crossings known) lies inside — instantiation, and it blocks saturation honestly — while an *uncharted* shape lies outside, labeled discovery, never a guess.

Definition 2.4 (Conditional routes)**.** A route may mix implemented and frontier hops. Its *conditional contract* is the meet over all hops — achieved contracts for implemented hops, required contracts for frontier hops — and it is read as: *if* each frontier hop is discharged by a pair achieving its requirement, this route answers q with at least this contract. Conditional routes are never executable; they are the plan-shaped reading of the map, and Theorem 5.7 is what makes them sound.

Definition 2.5 (Saturation)**.** B is *saturated at* G when the pair-shaped demand its open questions derive contains no *registered-able* candidate over the known set — no tier-2 design over the registered languages, solvers, and anchors would add a feasible way, a cheaper way, or a more trusted way — i.e., every question in B is in one of two terminal states: *solved, all ways*, its answer carrying the census of every feasible route with measured cost profile, assurance class, and corroboration; or *open, on the books*, its frontier pairs (if any are honest to name) all lying outside the known set, partitioned by obstacle. Saturation is an emptiness check on a derived set — mechanically detectable, no judgment call — and it is *reopened only by good news*: a new solver, anchor, or logic re-enters the known set exactly where the terminal board says it plugs in.

The map. The deliverable of saturating B is its *map*: the answered fraction of B per loop iteration (monotone, by Theorem 5.2); cost per answer across iterations; per solved question, the way-census; and the *terminal board* — the surviving frontier pairs with their obstacle partition, required contracts, design evidence, and distinct-question counts. The board is the domain's future-work section, machine-generated; the map is drawn in the same currency the instrument is built from.

3 The instrument, as means

None of hurdy-gurdy's load-bearing features was designed as a feature of one more verification tool. Read from the story's end, each is a requirement of frontier cartography — what it takes for the maps of Section 2 to be worth keeping:

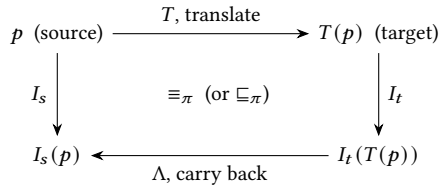

Figure 2. The pair as a commuting square: interpreting the source directly agrees, on the declared projection π, with translating, interpreting the translation, and carrying the behavior back. A directional (abstraction) pair declares $\sqsubseteq_\pi$ instead and ships a witness embedding along which the lax square is checked exactly; a failure localizes to a step and an observable.

a trustworthy map needs…	supplied by
labels its explorer cannot falsify	determinism + the square (§3.1)
composition told honestly, once	the contract meet (§3.2)
untrusted authors, trusted answers	the asymmetry (§3.3)
ground that is never lost	the gate and the ratchet (§3.4)
the far side, saved as evidence	the books, on two planes (§3.4)

3.1 Pairs and the directional square

The unit is the *pair*: two languages with formal semantics, a pure translator T, pure shared interpreters I_s, I_t, and a pure target-to-source interpreter Λ carrying target behaviors back to source vocabulary. These close a commuting square (Figure 2), $I_s(p) \equiv_\pi \Lambda(I_t(T(p)))$, up to a declared projection π — the observables the pair promises to preserve — and the square is *directional*: an abstraction pair declares $\sqsubseteq_\pi$ instead (every source behavior has a target counterpart; the target may deliberately have more) and ships a witness embedding along which its lax square is checked exactly like an exact one. The square is decidable *per program*: run both sides, compare under π, and a failure names its step and observable. That per-program decidability, resting on determinism (same input, byte-identical output, always), is what makes the map's solved region checkable rather than asserted: every entry can be re-derived, and a defect points at itself. Squares paste, so pairs compose into *routes* through the registry graph [12].

3.2 Contracts and the meet

A pair's composable declaration is its *contract*: assurance class, direction, kept observables, measured cost. Contracts compose along a route by the componentwise meet — the weakest hop on every axis at once, mechanized as Contract.comp_glb — which is the single composition statement the currency of Definition 2.3 needs: achieved and required contracts live in one lattice, and the meet is why a route's label never overstates. Two mechanisms move trust where the meet says it cannot go: per-run *re-establishment* by inline checking, and *branches* — independently derived routes to the same target whose agreement corroborates both, judged by their declared semantic artifacts, because two legs reading the same manual are one leg. Anchors, unlike pairs, exist in small finite supply; the trust axis saturates there, and the map records where (Section 2).

3.3 The asymmetry

One asymmetry organizes all trust in the platform, and it is why the explorer may be an untrusted LLM at all. An *existential* answer (a reachability witness) is self-certifying: the solver's model is carried back to a concrete input and replayed through the shared source interpreter — Λ proposes, I_s disposes — so a wrong or adversarial pair cannot forge it (existential_self_certifying, axiom-free). A *universal* answer is exactly where grades, branches, and independently re-checked certificates earn their cost, and along an over-approximating route it transfers soundly downward while existentials never rest on the route at all (lax_universal_transfer). A replay failure on an abstraction is a *spurious counterexample* — not a bug but a refinement demand, which is how CEGAR lives on the platform with its refinements as registered, reusable pairs (Section 4).

3.4 The gate, the ratchet, and the books

Admission is gated: a candidate pair arrives with its declared projection, direction, and grade, and clears a square-rebuilding grader that never runs pair code in its own process, two-sided negative controls (a seeded defect must be caught; the intact pair must pass), conjoined coverage against inventories its author cannot shrink, and coordinator-attested provenance. The *ratchet* then keeps every prior verdict standing: extensions preserve faithfulness on the old domain and coverage is monotone (ratchet_preserves_faithful, ratchet_coverage_mono) — F2's pair-level half. Everything else the theorems need is bookkeeping, and the platform keeps exactly one set of *books*: measured cost beside recorded demand, each demand a question verbatim with its first failing obstacle and named target, origin-tagged (*organic*/*campaign*/*scout*) and suite-tagged so a campaign cannot launder its own probing into evidence. The architecture is two planes meeting only in the registry and the books — the use plane reads declarations and produces evidence-carrying answers; the evolution plane turns demand into gated, ratcheted declarations — *answers never write; growth never answers* [12].

4 The loop

The explorer runs two loops with different payoffs (Figure 3). The *capability* loop is triggered by unanswerable questions: the diagnosis (Definition 2.2) names the first failing condition and, with it, a generation target — a missing pair, a wider projection, a missing reasoning language, a reduction, an independent anchor. The *trust* loop is triggered by answers

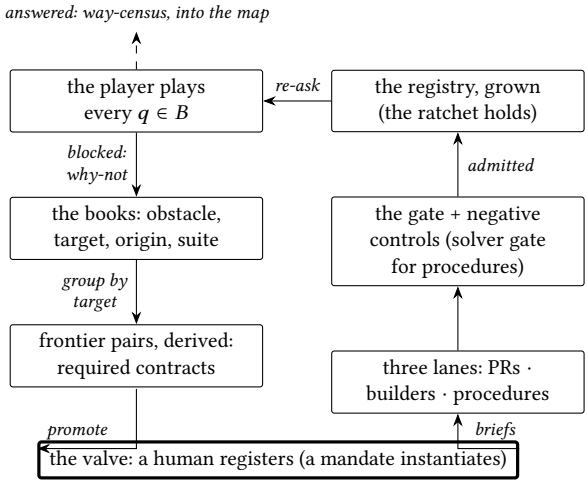

Figure 3. One loop iteration. Demand descends on the left — every failure a typed record on the books, aggregating into frontier pairs — and supply ascends on the right, but only through the valve at the bottom: registration is the one human act (or its earned, revocable mandate). The two planes meet nowhere else: answers never write, growth never answers. Monotonicity (Theorem 5.2) makes the cycle a ratchet, never a treadmill.

weaker than the asker's floor: a second route answers no new question at all, but an *independently derived* second route manufactures fidelity, and the trust advisor names the honest option — an existing independent branch, a route from a new semantic artifact, or saturation of the anchor supply, stated as such. Both loops consume the same books and produce the same currency: demand records that aggregate, per generation target, into the frontier objects of Definition 2.3.

Recommended, then registered. A pair must pay for itself by removing a named obstacle, so a pair is *recommended by its evidence* before it is registered: a brief opens by citing the demand behind it — mechanically pre-filled from the frontier object it promotes, required contract and citing questions verbatim. Registration itself stays a human act, the platform's one scope valve: the books recommend, volume is not a verdict, and a human decides what enters the queue. The delegated generalization is the *mandate*: a human registers not a pair but a region — a benchmark, the obstacle classes in scope, the admissible languages, the protected floors — within which demand-cited briefs may be registered mechanically, earned by the same shadow discipline as autonomous merging and revoked by the hand that wrote it. The valve does not move; what is delegated is its mechanical instantiation.

The two production lanes. Pairs are produced by others, through pull requests the gate checks exactly as it checks a builder agent's — green CI is designed to *mean* safe-to-merge — and by the platform's own builders, dispatched at partial pairs and unregistered briefs, integrated by a coordinator whose merge queue orders, fans out re-validation, and proposes; its autonomy is graduated rung by rung on a ledger of shadow trials, never granted once [12]. Both lanes feed the same registry under the same ratchet, which is what makes F2 a property of the system rather than of one workflow.

The third lane: decision procedures. The books can also demand what no route can supply — a decision procedure itself. The demand arises in exactly two places: a blocked shape the atlas locates in a known procedure family that no registered hub declares, and a scouting run in which every prototyped embedding measurably explodes, the blowup riding as the demand's evidence. A candidate procedure, however authored, enters through a *solver brief* mirroring the pair brief — declared shapes, a budget schema, a per-shape-and-verdict certificate obligation, and a *lineage* declaration, so that corroboration counts only agreement across disjoint lineages and a teacher's student cannot launder agreement into independence — and an admission gate that is the pair gate's analogue: replay of the corroborated solved region (any disagreement fails admission outright; agreement banks corroboration for every replayed question), canaries and verdict-flip mutants (a candidate that cannot be made to fail is not checked but unfalsifiable), certificate discipline, and budget honesty (*resource-out* at the gate is a recorded cost profile, not a failure). One inversion is the point: a synthesized in-process procedure is gated strictly *harder* than the pinned binaries beside it — sandboxed like untrusted pair code and determinism-checked, with no external-engine exemption — so the least-trusted authors produce the most-checkable artifacts. The lane sits deliberately behind the valve: procedure design is a creative act, no mandate rung exists for it, and under every mandate the demand escalates to a human. Its tooling is exercised end to end — the reference inhabitant is a hand-built bounded-enumeration engine admitted through the gate — and no more is claimed here: whether synthesized procedures move real frontiers is exactly a question the saturation reports are designed to answer.

CEGAR is the cost axis's engine. Against the cost condition, the one reduction that helps is the abstraction pair, and the loop it induces is counterexample-guided refinement with one structural difference from every CEGAR engine built inside a solver: the refinements are not throwaway internal state but registered, audited, reusable pairs. A spurious counterexample — an abstraction's witness that fails source replay — is a refinement demand on the books; the refined abstraction joins the graph and cheapens the next thousand questions. The abstraction advisor makes the dial mechanical (the question's cone of influence, the zero-loss free set, a farthest-first refinement ladder), and the report's

failure-mode reading (Section 6) says when refinement is the wrong remedy altogether.

The driver, and the structural valve. The loop's mechanical form is deliberately one iteration per invocation: pin the benchmark, play every question (the platform's own decide plumbing for hub-native questions; the general player plugs in at the same seam), book every spent verdict as a cost demand with an ascending probe so the blocked instance's cost curve is measured, re-diagnose, derive the board, deposit one self-contained iteration record, regenerate the report. The loop then *stops*: registration happens between invocations, by a human or a mandate, and the next invocation measures the growth. The human valve is structural, not a prompt — there is no code path from a campaign run to a registry write.

5 Why this explores the frontier

An explorer of a frontier must satisfy six properties or it is not an explorer: it must draw a map its untrusted explorer cannot falsify (F1), never lose ground (F2), know at every open point what would extend the map there (F3), eventually reach every point inside the true frontier (F4), know when a survey is finished (F5), and work in any territory (F6). This section states each as a theorem over the model of Section 2, with its proof burden placed honestly: mechanized, reduced to a named assumption, or empirical. The model is small on purpose — a registry is a set of items, a candidate is a list of items, and answerability is the filtration of Definition 2.1; its two monotonicities (stages shrink along conditions, grow along registries) carry F2–F5.

Theorem 5.1 (F1, unfalsifiable map)**.** *Whatever the generator of pairs does — blind or adversarial — the map's labels are truthful: (i) every existential answer is true at the source, assuming only adequacy of the source interpreter; (ii) every universal answer carries exactly the assurance class, direction, and trusted base computed by the componentwise meet of its route's contracts — labels never overstate, because meets never exceed their arguments.*

This is the instrument's own theory, consumed as an interface: (i) is witness replay (existential_self_certifying, axiom-free), (ii) is the contract algebra (weakest_link_universal, Contract.comp_glb, lax_universal_transfer) [12]. The residue is stated, not hidden: a false universal in the corner where no branch corroborates and no anchor covers is not excluded by theorem — it is bounded empirically (measured escape rates, negative controls) and shrunk by the trust loop, and the map records per answer which corner it sits in.

Theorem 5.2 (F2, monotone exploration)**.** *If $G \subseteq G'$ then every question answerable at G is answerable at G'* (Frontier.answerable_mono, axiom-free)*; and, by the pair-level ratchet* (ratchet_preserves_faithful)*, every standing verdict stands.*

Theorem 5.3 (F3, complete local gradient)**.** *For every question unanswerable at G: a first failing condition exists* (diagnosis_total)*, is unique* (diagnosis_unique)*, never regresses under growth* (diagnosis_progress)*, and strictly advances under any* adequate *extension — one after which some admitted candidate survives one condition further* (diagnosis_strict_progress).

Totality is where the model earns the word *complete*: the five conditions are exhaustive *by construction*, because answerability is defined as their conjunction (Definition 2.1), so the demand taxonomy is a theorem input, not a design choice. What the platform's diagnosis *names* — that the returned generation target, if admitted, discharges the failing condition — is the specification of `why_not`, met by construction in the implementation; it enters the theorems only as the adequacy hypothesis. The one classical step (diagnosis_total) is itself informative: an unanswerable question does not name its obstacle constructively — the platform computes it because its five conditions are decidable.

Theorem 5.4 (F4, the chain lemma)**.** *Along any growing chain of registries in which every diagnosis is met by an adequate extension, N steps answer the question* (adequate_chain_answerable)*: the diagnosis index is bounded by N and strictly increases, so it runs out of room.*

Assumption 1 (Fair enumeration)**.** Every demanded target is eventually attempted: the books aggregate demand, and a human — or a scoped mandate — eventually acts on standing demand.

Assumption 2 (Gate liveness)**.** A sound design with adequate evidence is eventually admitted: builders eventually produce, and the gate does not reject forever, a pair discharging a named requirement that some sound pair can discharge.

Theorem 5.5 (F4, relative completeness)**.** *Under Assumptions 1 and 2, every question answerable at some finite extension of G over the candidate universe is eventually answered by the loop.*

The mathematical content is Theorem 5.4; the assumptions supply its chain, and they are the theorem's honest trusted base — both are what the automation pipeline exists to provide, and both are measured (queue throughput, gate escape rates) rather than provable. The framing is deliberately that of relative completeness [7]: complete relative to the candidate universe and the domain's supply, not an oracle claim — undecidability, the adequacy floor of the source semantics, and the finite anchor supply stand at every registry size.

Theorem 5.6 (F5, saturation is a decidable, reachable fixpoint)**.** *For a pinned finite benchmark and a finite known set, the saturation predicate of Definition 2.5 is computable from the books — an emptiness check on the derived tier-2 set — and the loop restricted to in-set targets reaches it in finitely many*

iterations: target signatures are finitely many, an admitted signature cannot recur (dedup plus the ratchet), and each iteration retires at least one.

The finite-pool argument — admitted signatures never recur, and form a duplicate-free subset of the pool, so admissions run out of room — is mechanized as saturation_terminates. The decidability half is the implementation's contract: saturation is an exit code, not a judgment.

Theorem 5.7 (Currency: the status ratchet; conditional plans)**.** *(i) Along any lifecycle a pair's tier only advances and its evidence payload only accumulates* (lifecycle_ratchet, axiom-free)*: a built pair still carries the demand that created it. (ii) A mixed route's conditional contract (Definition 2.4) is a lower bound on its realized contract once every frontier hop is discharged by a pair whose achieved contract dominates its requirement* (conditional_plan_sound, via monotonicity of the meet)*: conditional plans never overpromise.*

Theorem 5.8 (F6, domain-genericity)**.** *F1–F5 are stated and proved over the domain signature alone — abstract languages, items, conditions, contracts — so they transfer to any domain by instantiation. What does not transfer for free is the supply side, and Section 6 states it as the explicit obligation: the theorems are conditional on the kit, and the books measure the condition.*

Necessity: the ablations. Each load-bearing feature of the instrument is needed for a specific theorem, and dropping it breaks that theorem — which is the precise sense in which the current platform is the means to this paper's end. Without *determinism*, a square check cannot distinguish defect from coin flip and F1(i)'s replay proves nothing. Without the *ratchet*, F2 fails and the curve can regress; a map that can silently shrink is not a map. Without *typed aborts* and the fixed diagnosis order, the filtration of Definition 2.1 has no well-defined first failure and F3 degenerates to blind enumeration. Without the *existential/universal asymmetry* — replay for witnesses, graded meets for universals — an untrusted generator forges map entries and F1 falls adversarially. Without *pinned ingestion*, "the benchmark" is not a fixed set and F5's fixpoint is not defined. And without the *human valve* (or its mandate generalization, which preserves it), Assumption 1's enumeration has no scope and the books no meaning: demand-driven growth presupposes someone accountable for what counts as demand.

The trusted base, collected. F1(i) assumes source-interpreter adequacy; F1(ii)'s corner is empirical; F4 assumes Assumptions 1 and 2; F5 assumes the pin and the finiteness of the known set; F6 defers the supply side to the kit. Nothing else is assumed, and each assumption is either measured by the platform's own books or discharged by construction in the implementation — the reader can locate every claim's burden, which is the property the whole architecture is built to have.

6 The domain kit, and the far side as a design oracle

6.1 The kit

Pointing the explorer at a domain costs exactly four things, and the theorems of Section 5 are conditional on them — this list is the instantiation obligation of F6, frozen. Each item is named, so that this paper and the platform's design documents cite one vocabulary; and each is stated with what its absence costs, so a domain can read off exactly which guarantee it forfeits by not supplying it:

- **K1 A reasoning hub**: at least one language a mechanized decision procedure consumes directly, with declared question shapes and declared budgets. (Without it, no question clears the shape and cost conditions.)
- **K2 Deterministic shared interpreters** for every language touched, exposing named observables. (Without them, squares cannot be checked and F1's replay proves nothing.)
- **K3 At least one external semantic anchor**, if trust beyond per-run checking is wanted. (Without it, the trust axis saturates at *checked*; the theorems still hold, and the map says so.)
- **K4 A pinned benchmark**: a finite question set with recorded provenance. (Without the pin, F5's fixpoint is not defined.)

K3 is the one graded *optional*, and the shape of its grading is the kit's own discipline in miniature: an absent optional obligation is a *recorded ceiling*, never a silent failure.

The kit is deliberately mundane: hardware model checking supplies all four today; C program analysis supplies them down a longer spine; and the platform's own field-blind corners (chemical reaction networks, molecular notation) are the cheap existence proof that nothing in the list assumes computation-adjacent languages [12].

6.2 The far side as a design oracle

A terminal board is not a failure list. By the honest-failure discipline, every open question carries typed evidence, and each obstacle class comes with an *extraction operator* turning that evidence into the specification of the missing instrument:

obstacle	evidence	extraction operator
shape	blocked φs verbatim	fragment hull + atlas
charted shape	hull + census slice	procedure-synthesis brief
cost	cost curves at the wall	binding-parameter fit
cost (abstraction)	the spurious corpus	separating predicates
trust	uncorroborated verdicts	certificate + anchor joins
outside the closure	the wall itself	in-closure relaxation

Four operators are already mechanical. The *binding-parameter fit* is the report's failure-mode reading: blocked instances get their cost curve measured by an ascending probe, and the fit classifies — exponential in the unrolling

bound means deeper search will not close it (demand an unbounded engine or a property transformation); linear means depth is affordable (raise the budget before demanding an instrument). The *atlas location* is a registry-side reference table of the known decidability landscape: a shape-blocked demand arrives locating itself — the setting in which its shape is decidable, the procedure family that decides it natively, and the *known crossing*, the classical reduction that would discharge it with an endo-pair on an existing hub instead of a new logic (liveness-to-safety for liveness; self-composition for 2-safety hyperproperties; monitor weaving for temporal shapes). The atlas is reference data, protected like every other instrument the gate trusts, and a shape it does not know reads *uncharted*, never a guess — and it is load-bearing: chartedness is what classifies a procedure demand as instantiation (inside the known set, blocking saturation) or discovery (outside, on the frontier). The *scout* measures before anyone designs: candidate embeddings for a blocked shape are prototyped and their encoding blowup measured, so a procedure demand arrives justified by numbers rather than taste. And the *procedure-synthesis brief* turns a charted, procedure-shaped demand into a self-contained work item — the fragment hull of its citing φs, the atlas location and family, the required contract, the census slice that will falsify the build, and the certificate obligation. The recommender is audited in kind: every recommendation is a prediction of closure, checked ex post against realized closure. The remaining operator (separating predicates mined from the persistent spurious corpus) is named work, staged behind the same shadow discipline as every other delegated judgment.

Challenge bundles. Frontier pairs are exportable: a board entry carries pinned citing questions, declared budgets, a required contract, and the near side's baseline census — a self-contained challenge for the domain's solver community. This is the sense in which the map's far side is a demand oracle for the ecosystem: competitions supply benchmarks to the platform; terminal boards supply open problems, with evidence counts, back.

6.3 The pre-registered protocol

This paper was written with no benchmarks section, its place held by the following protocol, pre-registered before any campaign ran — which is what keeps the report honest. Section 7 is the saturation report the protocol produced.

The benchmark. The hardware model-checking competition corpus (HWMCC [22]): designed by others, labeled, native to the platform's bit-level hub — no translation debt to pay before the loop starts. Ingestion is streamed-with-pin from a community mirror at a fixed commit, every instance sha256-pinned; the campaign starts from the already-pinned slice and widens by families, one pinned extension at a time. The natural second act is software verification (SV-COMP [3]) down the full C-to-solver spine.

The protocol. (1) Pin the suite; establish ground truth where labels exist. (2) Run the loop's player over every question, books on, *origin=campaign*, suite-tagged — manufactured demand is displayed apart from organic demand by construction. (3) What answers, answers with its way-census; what does not lands on the books with its first failing obstacle, and blocked instances get their cost curves measured. (4) Derive the board; promote what a human (or, when earned, a mandate) registers; build, gate, merge. (5) Re-run. The ratchet makes the curve monotone; stop at the fixpoint, which is an exit code.

Declared caps. Unrolling bound, wall-clock and memory per decide, instance count per iteration, one instance in memory at a time; all caps ride in every result's provenance, and capped results are labeled capped, never implied full-suite.

Pre-registered honesty. The curve may plateau on cost; the plateau is a finding — the measured practice-frontier of the domain — not a failure. *unknown* and *resource-out* are counted, never hidden. A terminal board dominated by frontier-only entries is a *successful* saturation. The tooling (the loop driver, the saturation checker, the report generator, the promotion command) is built, tested, and has been exercised end to end on the pinned slice; no number from any run appears in this paper.

7 The saturation report

The protocol of Section 6.3 has now run: six loop iterations over one pinned suite, deposited iteration by iteration with the books on. This section is the report the protocol promised. Every number below is read from the deposited ledger; regenerating the report from the same ledger is byte-identical, and the caps of every iteration ride in every record's provenance.

The pin. The suite, `hwmcc-sosylab-beem`, is the first widening of the pre-registered slice: the same community mirror (`CyanoKobalamyne/ hwmcc-benchmarks`) at the same commit (`57174f5`), widened to the two labeled-adjacent bit-vector families `bv/2024/sosylab` and `bv/2019/beem` — 110 reachability questions, every instance sha256-pinned, fetched streamed-with-pin one at a time and hash-verified against the pin. Five instances carry external labels inherited from the hand-pinned slice (four *unreachable*, one *reachable*); the rest earn ground truth the protocol's way — engine agreement and witness replay, never assumption. Across all six iterations, every verdict agreed with every standing label, and no disagreement event was ever booked.

The campaign.

it.	decision surface (declared caps)	ans.	open	decide (s)	s/ans.
0	btormc, k=20, probes {2, 4, 8}, 300 s	79	31	6706	84.9
1	+ btor2-havoc, CEGAR ≤4 rounds	79	31	5321	67.4
2	same (the map moved, not the caps)	79	31	5270	66.7
3	+ pono, 2 unbounded modes, 300 s each	82	28	4209	52.6
4	pono, 3 unbounded modes, 600 s each	80	30	4194	52.4
5	+ avr; 6 exploration modes, 600 s each	79	31	23629	299.1

Iteration 0, the baseline. The native hub alone answers 79 of 110; the 31 *resource-out* questions cluster onto a single in-set frontier entry, and the ascending probe measures their cost curves.

Iterations 1–2, the reduction. The promoted btor2-havoc abstraction played and spent: of the 31 blocked questions, 27 reach the declared four-round CEGAR cap with every counterexample spurious, 3 wall out mid-refinement, 1 stops at round one; localization has no free room anywhere in the cluster (free_havoc = 0). The curve stays flat — a finding, per the pre-registration, not a failure. In iteration 2 the map moved while the numbers held: the board's one entry advanced to d4c59dafc402 [in-set] *native-procedure* — BMC / k-induction / IC3-class model checking on the solver hub — with the reduction cited as played-and-spent. The books demanded the unbounded engine before anyone built it, and the binding-parameter fits (below) predicted that deeper bounded search could not close the cluster.

Iterations 3–4, the engine the books demanded. pono enters through a solver brief and its admission gate. Iteration 3 moved the curve for the first time in three iterations and put the first *unreachable at every depth* verdict on the books: Problem11_label26, proved by k-induction in 74 s after ic3bits spent its wall — exactly the closure the exponential-in-k fits said bounded search would never reach. The iteration's +3 decomposes honestly as 1 proved + 2 unmeasured (their fetches went offline mid-run; booked, not hidden). Between iterations, one versioned budget amendment: the unbounded wall raised to 600 s per mode, the portfolio widened to three modes (ind moved first, then ic3bits, mbic3), and the amended declaration re-admitted by the gate before it played. Iteration 4 then spent the full amended budget — all 30 blocked pins through all three walls, 1800 s each, ~15 h of declared spend — and closed nothing new: the two offline pins came back measured as blocked (the honest dip in the curve), and Problem11_label26 was re-proved cheaper under the reordering (85.9 s with ind first, against 374.3 s behind ic3bits). Tripling the unbounded attention closed zero new pins: the strongest saturation evidence the books had yet held.

Iteration 5, the exploration. Every family member the campaign had never played, against every blocked pin: avr first — host-built, its (avr, yices) lineage disjoint from both btormc's and pono's, so its agreement is the first cross-lineage corroboration an unbounded verdict here could earn — then pono's interpolation, abstraction-refinement, and synthesis-guided sub-families (dar, interp, ismc, ic3ia, ic3sa, sygus-pdr). All 31 portfolio runs spent, ~3600 s each (five full 600 s walls and two fast abstentions), with already-spent modes cited from the books rather than re-played; zero closures among the standing blocked. The catches came at the wall's edge instead: gcd_4+newton_3_3, bounded-answered near the cap in iteration 4, walled out this time, fell into the portfolio, and dar proved it unreachable at every depth (1376.6 s — the campaign's second unbounded closure, the interpolation sub-family's first); gcd_2+newton_3_7 likewise fell in and answered *reachable* with a witness that replays through the shared interpreter (1492.6 s). avr did not corroborate Problem11_label26: it abstained — replay-gated, honest — rather than contradict, and the corroboration remains open.

Reading the curve. The answered column is each iteration's portfolio coverage under that iteration's declared caps — not the platform's cumulative knowledge, which is the quantity Theorem 5.2 makes monotone, and which never receded: 80 of 110 questions answered at least once, two of them at every depth. The iteration-4 dip is iteration 3's +3 decomposing under measurement; the return to 0.7182 in iteration 5 prices the exploration portfolio itself — Problem11_label26 books *resource-out* there because its closer was deliberately not re-played, and its iterations 3–4 proof stands on the books. Cost per answer fell from 84.9 s to 52.4 s as the surface sharpened; iteration 5's 299.1 s is the declared price of exhausting a family enumeration, not of answering. *resource-out* and *unmeasured* are counted throughout, never hidden.

What answered, and with what trust. The final census: 22 *reachable* — a reachable verdict here carries a witness, and witnesses replay through the shared interpreter — 57 *unreachable* at the declared bound, priced as bounded claims, and 31 *resource-out*. Every answered question's way-census holds both registered ways (btor2-smtlib and native), universal and exact. The two at-every-depth closures are engine-answered: their briefs type the certificate obligation open until invariant re-discharge, and neither has yet earned cross-lineage corroboration. The map prices trust; it does not launder it.

The terminal board, and what did not saturate. The board holds one entry: d4c59dafc402 [in-set] *native-procedure*, 31 distinct citing questions, every one *origin=campaign* — manufactured demand displayed apart from organic, by construction. The failure-mode reading over the books' measured systems: 8 fit exponential-in-k (deeper bounded search will not close them — the prediction iterations 3 and 5 each confirmed with a closure), 46 flat, 173 unmeasured (the fit demands at least three distinct bounds before it speaks). The entry is *in-set*: the atlas charts the demanded family, so this is instantiation, not discovery — and by Definition 2.5 an in-set entry blocks

saturation honestly. The mechanical check agrees: the loop's final exit code says *not saturated*. What terminated is the host's enumeration: BMC, k-induction, bit-level / model-based / equality-abstraction IC3, interpolation, abstraction-refinement IC3, synthesis-guided PDR, and the havoc reduction have all been played and spent against the blocked cluster, and the members not yet played need what this host cannot hold — IC3IA over MathSAT behind a Linux-only binary, and ABC behind an AIGER pair and two builds. The deposit is therefore the measured practice-frontier of this host's known set, and the board entry is exportable as a challenge bundle: pinned citing questions, declared budgets, a required contract, the census. Saturation is reopened only by good news, and the board says exactly where the good news plugs in.

Deviations from the pre-registration. Three, all recorded where they happened: the versioned budget amendment between iterations 3 and 4, re-admitted by the gate before it played; the two mid-run offline fetches in iteration 3, booked *unmeasured* and measured in iteration 4; and iteration 5's portfolio composition — spent closers cited from the books rather than re-played — declared in its caps and read accordingly above.

The maps compound: the engines, briefs, and gate discipline this campaign forced into existence are already registered instruments for the protocol's named second act, software verification down the full C-to-solver spine.

8 Related work

Decidability by reduction. Answering a question by carrying it to a decision procedure is a classical discipline. In logic it runs from quantifier elimination [26, 30] through interpretations that transfer decidability between theories [31] to reduction into expressive decidable hubs — Rabin's S2S, into which many decidable theories interpret [27] — and the composition method [17]; algorithmic meta-theorems [8] and well-structured transition systems [11] decide whole logically-delimited families at once. In verification practice the same move is institutional: bounded model checking reduces to SAT [5], theory combination composes decision procedures [21], intermediate verification languages pipeline programs to SMT [2, 10], and Ivy makes landing in a decidable fragment the design objective itself [23, 24]. Each strand fixes the reduction — one interpretation, one pipeline, one fragment — and asks what it decides. Answerability (Definition 2.1) makes the same move an object of study: the answerable set is the closure of a domain's questions under an evolving, gated registry of reductions, budgeted cost sits inside the definition rather than after it, and the deliverable is the boundary — every open point carrying its first failing obstacle and the instrument that would move it.

Single-edge theories. Certified compilation [13, 15] and translation validation [16, 20, 25] are statements about one translation; the instrument paper [12] develops the graph-level theory this paper consumes. What is new here is neither the edges nor the graph but the *use*: the graph run as an explorer whose product is the boundary itself.

Abstraction and CEGAR. Abstract interpretation [9] supplies the lax half of the directional square, and counterexample-guided refinement [6, 14] the cost loop's shape. The structural difference is bookkeeping with consequences: every refinement is a registered, audited, reusable pair, and every spurious counterexample persists on the books as evidence for the next abstraction's required contract, rather than dying inside one solver run.

Certifying answers. Proof-carrying code [19], certifying algorithms [18], verifiable witnesses [4], and checked proof formats [1, 29, 32] supply the platform's trust vocabulary. The frontier reading adds that the *absence* of a certificate is itself data: trust-blocked questions aggregate into capability demands naming the missing checker or anchor. The same discipline is what makes the procedure lane safe to open at all: a candidate decision procedure is never trusted — its certificates are checked and its answers replayed against the corroborated census — so admitting untrusted, even LLM-written, procedures does not dilute what an answer means (Section 4).

Competitions and infrastructure. HWMCC [22], SV-COMP [3], and execution infrastructure like StarExec [28] supply pinned corpora, labels, and the solver ecosystem's measured state of the art — they rank solvers on fixed benchmarks. Saturation inverts the roles: the benchmark is fixed and the *instrument grows* until the benchmark is exhausted, and the residue — the terminal board, exported as challenge bundles — is a machine-generated demand side for the next competition's entrants.

Relative completeness. The framing of F4 is deliberately Cook's [7]: completeness relative to an oracle for what cannot be internalized — here, the candidate universe, builder competence, and the domain's supply, each named as an assumption and each measured by the platform's own books rather than assumed silently.

9 Limitations and conclusion

Four walls stand at every registry size, and the theorems are drawn up to them, not past them. *Computability*: a route re-expresses a question, never changes its degree; unbounded universal questions rest on bounded claims under declared bounds. *The adequacy floor*: no pair can make a question about something the source semantics does not define into a question about the program. *Cost*: deciding is exponential in

the worst case everywhere that matters, budgets are permanent verdict classes, and a plateau in the curve is a finding — the measured practice-frontier of the domain — not a failure. *Anchor supply*: capability scales with generated pairs; trust scales with independent formalizations of real semantics, which exist in small finite supply. Within the walls, the claim of this paper is deliberately modest in kind and, we believe, useful in consequence: an architecture whose exploration is sound, monotone, self-directing, relatively complete, and terminating per benchmark — provably, with the assumptions priced — turns "we don't know how to decide this" from a shrug into a typed, clustered, cost-profiled, evidence-citing specification of the missing instrument.

The maps compound, because pairs are shared: every benchmark starts on the ground the last one won. The near-term work is scale — widening the pinned corpus family by family, the software-verification spine as the second act — and the mandate rung, so that saturation's mechanical middle runs at machine speed between two human acts: writing the mandate, and reading the map. Two extensions are staged beyond it, at deliberately different maturities, both demand-driven. The synthesis lane — the books demanding decision procedures, a gate falsifying candidates, synthesized engines gated harder than the binaries they would join — is tooled and shadow-first; whether it moves real frontiers is a finding the saturation reports will or will not deliver, and we claim nothing sooner. And on the trust axis, demanding *mechanical proofs* of a pair's square through the same books — a certificate demand priced against an independent second route, translation validation per run or refinement proofs where a mechanized semantics exists — is designed, not built. The vision is the means. The map is the end.

Acknowledgments

This work was co-funded by the Czech Science Foundation under Grant No. 23-07580X and the European Union under the project Robotics and Advanced Industrial Production (reg. no. CZ.02.01.01/00/22_008/0004590).

References

[1] Bruno Andreotti, Hanna Lachnitt, and Haniel Barbosa. 2023. Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. In *TACAS*.

[2] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. 2005. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *Formal Methods for Components and Objects (FMCO) (LNCS, Vol. 4111)*. 364–387.

[3] Dirk Beyer. 2021. Software Verification: 10th Comparative Evaluation (SV-COMP 2021). In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 12652)*. 401–422.

[4] Dirk Beyer, Matthias Dangl, Daniel Dietsch, Matthias Heizmann, and Andreas Stahlbauer. 2015. Witness Validation and Stepwise Testification across Software Verifiers. In *ESEC/FSE*.

[5] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. 193–207.

[6] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *CAV (LNCS, Vol. 1855)*. 154–169.

[7] Stephen A. Cook. 1978. Soundness and Completeness of an Axiom System for Program Verification. *SIAM J. Comput.* 7, 1 (1978), 70–90.

[8] Bruno Courcelle. 1990. The Monadic Second-Order Logic of Graphs. I. Recognizable Sets of Finite Graphs. *Information and Computation* 85, 1 (1990), 12–75.

[9] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *POPL*. 238–252.

[10] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3 — Where Programs Meet Provers. In *ESOP*.

[11] Alain Finkel and Philippe Schnoebelen. 2001. Well-Structured Transition Systems Everywhere! *Theoretical Computer Science* 256, 1–2 (2001), 63–92.

[12] Christoph Kirsch. 2026. Untrusted Authors, Trusted Answers: A Calculus of Fidelity-Graded Translations. arXiv preprint arXiv:2607.14137, v2. Code, evidence, and the Lean mechanization: https://github.com/cksystemsgroup/hurdy-gurdy (v2 is repository tag arxiv.2).

[13] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *POPL*.

[14] Robert P. Kurshan. 1994. *Computer-Aided Verification of Coordinating Processes: The Automata-Theoretic Approach*. Princeton University Press.

[15] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115.

[16] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*.

[17] Johann A. Makowsky. 2004. Algorithmic Uses of the Feferman–Vaught Theorem. *Annals of Pure and Applied Logic* 126, 1–3 (2004), 159–213.

[18] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161.

[19] George C. Necula. 1997. Proof-Carrying Code. In *POPL*.

[20] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *PLDI*.

[21] Greg Nelson and Derek C. Oppen. 1979. Simplification by Cooperating Decision Procedures. *ACM Transactions on Programming Languages and Systems* 1, 2 (1979), 245–257.

[22] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *CAV*.

[23] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. 2017. Paxos Made EPR: Decidable Reasoning about Distributed Protocols. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 108:1–108:31.

[24] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. 2016. Ivy: Safety Verification by Interactive Generalization. In *PLDI*. 614–630.

[25] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *TACAS*.

[26] Mojżesz Presburger. 1929. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt. In *Comptes Rendus du I Congrès de Mathématiciens des Pays Slaves*. 92–101.

[27] Michael O. Rabin. 1969. Decidability of Second-Order Theories and Automata on Infinite Trees. *Trans. Amer. Math. Soc.* 141 (1969), 1–35.

[28] Aaron Stump, Geoff Sutcliffe, and Cesare Tinelli. 2014. StarExec: A Cross-Community Infrastructure for Logic Solving. In *Automated Reasoning (IJCAR) (LNCS, Vol. 8562)*. 367–373.

[29] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. 2021. cake_lpr: Verified Propagation Redundancy Checking in CakeML. In

TACAS.

[30] Alfred Tarski. 1951. *A Decision Method for Elementary Algebra and Geometry* (2nd ed.). University of California Press.

[31] Alfred Tarski, Andrzej Mostowski, and Raphael M. Robinson. 1953. *Undecidable Theories*. North-Holland.

[32] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *SAT*.